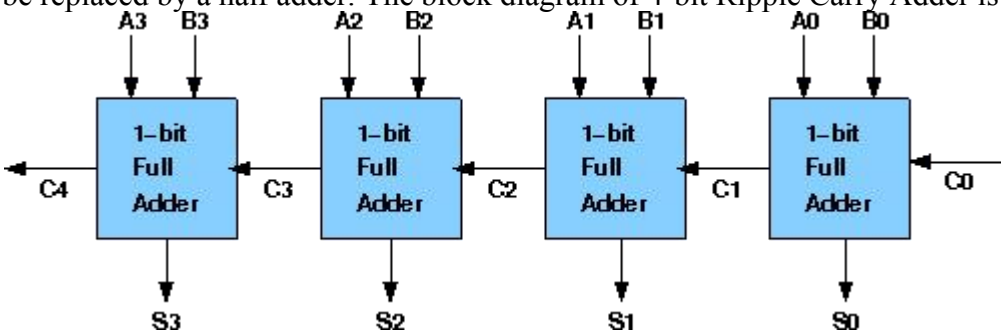
Aim: Write the Verilog code for designing 4 bit ripple carry adder circuit and verify.

IDE: EDA Playground

Theory:

Arithmetic operations like addition, subtraction, multiplication, division are basic operations to be implemented in digital computers using basic gates like AND, OR, NOR, NAND etc. Among all the arithmetic operations if we can implement addition then it is easy to perform multiplication (by repeated addition), subtraction (by negating one operand) or division (repeated subtraction).

Half Adders can be used to add two one bit binary numbers. It is also possible to create a logical circuit using multiple full adders to add N-bit binary numbers. Each full adder inputs a **Cin**, which is the **Cout** of the previous adder. This kind of adder is a **Ripple Carry Adder**, since each carry bit "ripples" to the next full adder. The first (and only the first) full adder may be replaced by a half adder. The block diagram of 4-bit Ripple Carry Adder is shown here below -



The layout of ripple carry adder is simple, which allows for fast design time; however, the ripple carry adder is relatively slow, since each full adder must wait for the carry bit to be calculated from the previous full adder.

Working of 4-bit Ripple Carry Adder

- Let's take an example of two input sequences 0101 and 1010. These are representing the A4 A3 A2 A1 and B4 B3 B2 B1.
- As per this adder concept, input carry is 0.
- When Ao & Bo are applied at 1st full adder along with input carry 0.
- Here A1 =1 ; B1=0 ; Cin=0
- Sum (S1) and carry (C1) will be generated as per the Sum and Carry equations of this adder. As per its theory, the output equation for the Sum = A1⊕B1⊕Cin and Carry = A1B1⊕B1Cin⊕CinA1
- As per this equation, for 1st full adder S1 =1 and Carry output i.e., C1=0.
- Same like for next input bits A2 and B2, output S2 = 1 and C2 = 0. Here the important point is the second stage full adder gets input carry i.e., C1 which is the output carry of initial stage full adder.
- Like this will get the final output sequence (S4 S3 S2 S1) = (1 1 1 1) and Output carry C4 = 0
- This is the addition process for 4-bit input sequences when it's applied to this carry adder.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Digital Design using Verilog (01CT0619)	**Aim: Write the Verilog code for designing 4 bit ripple carry adder circuit and verify.**	
Experiment No: 05	**Date:**	**Enrollment No: 92301733048**

Pre Lab Exercise:

a. What is Ripple Carry Adder?
A Ripple Carry Adder (RCA) is a combinational circuit used to add multi-bit binary numbers by connecting multiple full adders in series. Each full adder adds corresponding bits of two numbers along with a carry from the previous stage. The carry output from one stage becomes the carry input of the next stage. It is simple in design and widely used for binary addition in digital systems.

b. Why word Ripple is used in this combinational circuit?
The word "Ripple" is used because the carry output generated at each stage must propagate (ripple) through each full adder before the final result is obtained. The carry moves sequentially from the least significant bit (LSB) to the most significant bit (MSB), causing a delay in producing the final sum.

c. What is difference between "Ripple Carry Adder" and "Carry look-ahead generator"?
A Ripple Carry Adder generates carry sequentially from one stage to the next, which causes delay because each full adder must wait for the previous carry output. In contrast, a Carry Look-Ahead Generator (CLA) calculates carry signals in advance using Boolean logic, reducing propagation delay and increasing speed. Ripple Carry Adder has simple design but slower operation, whereas Carry Look-Ahead Adder has more complex design but much faster performance.

Verilog Code:

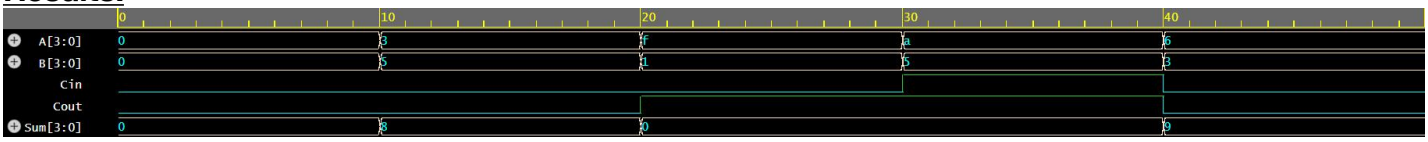
```
module full_adder(
input A, B, Cin,
output Sum, Cout
);

assign Sum = A ^ B ^ Cin;
assign Cout = (A & B) | (Cin & (A ^ B));

endmodule

module ripple_carry_adder_4bit(
input [3:0] A, B,
input Cin,
output [3:0] Sum,
output Cout
);

wire c1, c2, c3;

full_adder FA0(A[0], B[0], Cin, Sum[0], c1);
full_adder FA1(A[1], B[1], c1, Sum[1], c2);
full_adder FA2(A[2], B[2], c2, Sum[2], c3);
full_adder FA3(A[3], B[3], c3, Sum[3], Cout);

endmodule
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Digital Design using Verilog (01CT0619)	**Aim: Write the Verilog code for designing 4 bit ripple carry adder circuit and verify.**	
Experiment No: 05	**Date:**	**Enrollment No: 92301733048**

Test-bench:

```
// Code your testbench here
// or browse Examples
module tb_ripple_carry;

reg [3:0] A, B;
reg Cin;

wire [3:0] Sum;
wire Cout;

ripple_carry_adder_4bit uut(
.A(A),
.B(B),
.Cin(Cin),
.Sum(Sum),
.Cout(Cout)
);

initial
begin

$dumpfile("rca4.vcd");
$dumpvars(0, tb_ripple_carry);

// Test cases
A = 4'b0000; B = 4'b0000; Cin = 0; #10;
A = 4'b0011; B = 4'b0101; Cin = 0; #10;
A = 4'b1111; B = 4'b0001; Cin = 0; #10;
A = 4'b1010; B = 4'b0101; Cin = 1; #10;
A = 4'b0110; B = 4'b0011; Cin = 0; #10;

$finish;

end
endmodule
```

Results:



Observation and Result Analysis:

In this experiment, a 4-bit Ripple Carry Adder was successfully designed using Verilog by cascading four full adders. The carry output of each stage was given as input to the next stage, demonstrating the ripple carry effect. The simulation results verified correct addition of 4-bit binary numbers along with proper carry generation. This experiment shows that ripple carry adders are simple to design but introduce propagation delay due to sequential carry transfer.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Digital Design using Verilog (01CT0619)	**Aim: Write the Verilog code for designing 4 bit ripple carry adder circuit and verify.**	
Experiment No: 05	**Date:**	**Enrollment No: 92301733048**

Post Lab Exercise:

a. Write verilog code, test bench and output for carry look ahead generator.

Verilog Code:

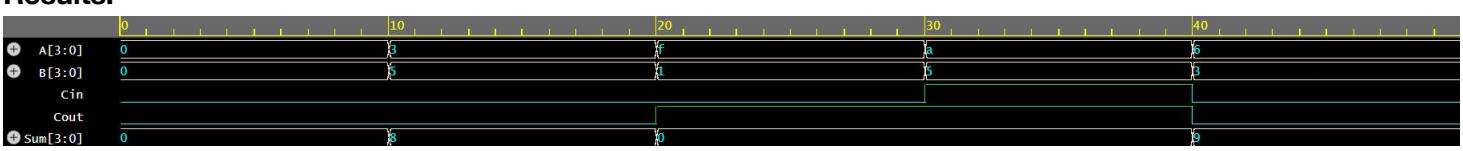
```
// Code your design here
module carry_lookahead_adder(
input [3:0] A, B,
input Cin,
output [3:0] Sum,
output Cout
);

wire [3:0] G, P;
wire C1, C2, C3;

// Generate and Propagate
assign G = A & B;
assign P = A ^ B;

// Carry equations
assign C1 = G[0] | (P[0] & Cin);

assign C2 = G[1] | (P[1] & G[0]) | (P[1] & P[0] & Cin);

assign C3 = G[2] | (P[2] & G[1]) | (P[2] & P[1] & G[0]) | (P[2] &
P[1] & P[0] & Cin);

assign Cout = G[3] | (P[3] & G[2]) | (P[3] & P[2] & G[1])
            | (P[3] & P[2] & P[1] & G[0])
            | (P[3] & P[2] & P[1] & P[0] & Cin);

// Sum
assign Sum[0] = P[0] ^ Cin;
assign Sum[1] = P[1] ^ C1;
assign Sum[2] = P[2] ^ C2;
assign Sum[3] = P[3] ^ C3;

endmodule
```

Testbench:

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Digital Design using Verilog (01CT0619)	**Aim: Write the Verilog code for designing 4 bit ripple carry adder circuit and verify.**	
Experiment No: 05	**Date:**	**Enrollment No: 92301733048**

```verilog
// Code your testbench here
// or browse Examples
module tb_cla;

reg [3:0] A, B;
reg Cin;

wire [3:0] Sum;
wire Cout;

carry_lookahead_adder uut(
.A(A),
.B(B),
.Cin(Cin),
.Sum(Sum),
.Cout(Cout)
);

initial
begin

$dumpfile("cla.vcd");
$dumpvars(0, tb_cla);

// test cases
A=4'b0000; B=4'b0000; Cin=0; #10;
A=4'b0011; B=4'b0101; Cin=0; #10;
A=4'b1111; B=4'b0001; Cin=0; #10;
A=4'b1010; B=4'b0101; Cin=1; #10;
A=4'b0110; B=4'b0011; Cin=0; #10;

$finish;

end

endmodule
```

Results:



b. Write verilog code in behavioral modeling technique for 2 bit ripple carry adder.

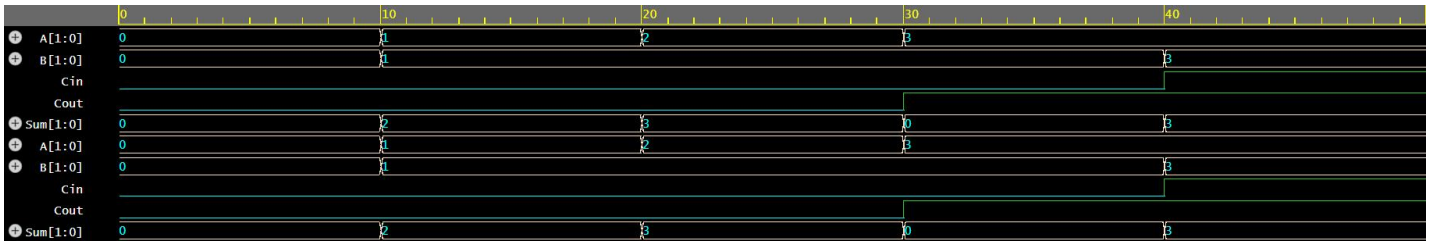
Verilog Code:

design.sv

```verilog
// Code your design here
module rca_2bit_behavioural(
    input [1:0] A, B,
    input Cin,
    output [1:0] Sum,
    output Cout
);

assign {Cout, Sum} = A + B + Cin;

endmodule
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Digital Design using Verilog (01CT0619)	**Aim: Write the Verilog code for designing 4 bit ripple carry adder circuit and verify.**	
Experiment No: 05	**Date:**	**Enrollment No: 92301733048**

Testbench:

```
// Code your testbench here
// or browse Examples
module tb_rca_2bit;

reg [1:0] A, B;
reg Cin;

wire [1:0] Sum;
wire Cout;

rca_2bit_behavioural uut(
.A(A),
.B(B),
.Cin(Cin),
.Sum(Sum),
.Cout(Cout)
);

initial
begin

$dumpfile("rca2.vcd");
$dumpvars(0, tb_rca_2bit);

// test cases
A=2'b00; B=2'b00; Cin=0; #10;
A=2'b01; B=2'b01; Cin=0; #10;
A=2'b10; B=2'b01; Cin=0; #10;
A=2'b11; B=2'b01; Cin=0; #10;
A=2'b11; B=2'b11; Cin=1; #10;

$finish;
end
endmodule
```

Results: